
CS121 Data Structures in C

01 Introduction & C Quick Review

Victor Junqiu WEI

FIE, MUST

Teaching Staff's Information

- **Dr. Victor Junqiu WEI (Lecturer-in-Charge)**
 - ◆ Email: jqwei@must.edu.mo
 - ◆ Phone: 3444
 - ◆ Office: A312
 - ◆ Office Hour: By Appointment

Textbook and Reference

- Textbook

Data Structures and Algorithm Analysis in C, Mark Allen Weiss

- ◆ Latest version = 2nd Edition
- ◆ Source codes are available online

- References

Data Structures : A Pseudocode Approach with C,
Richard F. Gilberg and Behrouz A. Forouzan

- ◆ Latest version = 2nd Edition

Textbook and Reference (cont'd)

- References (cont'd)

Mastering Algorithms with C, Kyle Loudon

- ◆ 1st Edition
- ◆ Source codes are available online

Introduction to Algorithms, Thomas H. Cormen,
Charles E. Leiserson, Ronald L. Rivest, Clifford Stein

- ◆ 2nd Edition

Grading Scheme

- Grading is based on
 - ♦ Class Participation 10%
 - ♦ Assignments 35%
 - ♦ Final Exam 55%
- Plagiarism Policy
 - ♦ Assignments: get 0
 - ♦ Final Exam: automatic FAIL

Grading Scheme

No late submission!

Number of Late Days	Deduction (out of 100 marks)
1	-10
2	-20
3	-30
4	-40
5	-50
5 above	0

Course Overview

- A fundamental computer science course
 - ♦ Essential for programming
 - ♦ Essential for advanced courses
- Course Prerequisite
 - ♦ Computer Language and Programming (C language)
 - Need to know C under command prompt environment (Unix/Dos) or any integrated development environment (VC)
 - Good programming skills
 - Translate pseudo-codes into codes
 - ♦ Basic mathematical skills
 - Computing complexity, solving recursive equations, etc.
 - ♦ Computer architecture
 - Pointers, storage, memory access, etc.

Course Outline

- C review
- Algorithmic asymptotic analysis
 - ♦ Big-O
- Lists, stacks, and queues
- Sorting
- Tree, search trees
 - ♦ Binary search tree
 - ♦ AVL tree
- Graphs
- Hashing
- Advanced algorithm (optional)

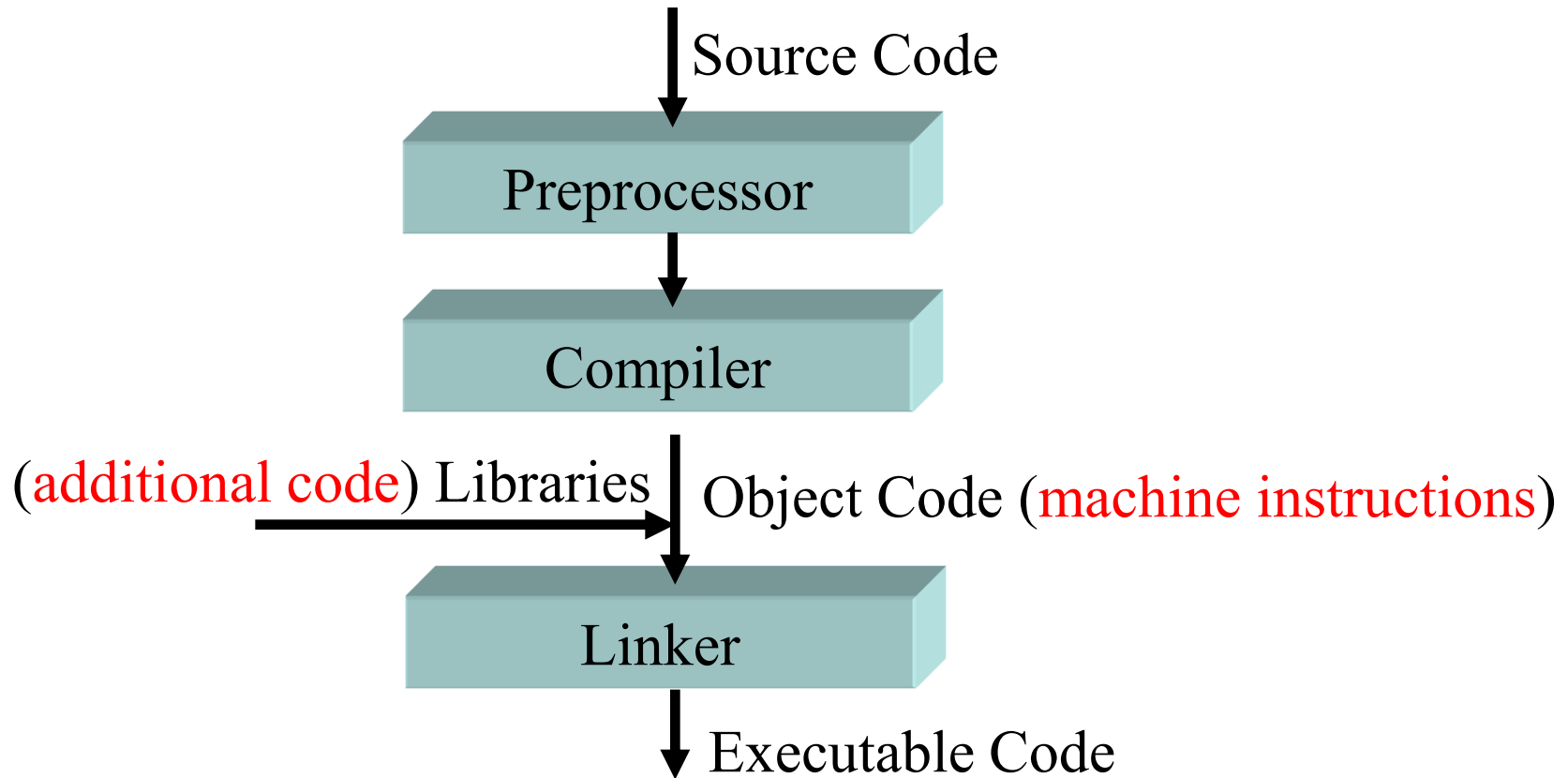
Goal of the Course

- Learn to design appropriate data structures
- Learn to solve problems using proper data structures and algorithms
- To make the program efficient

C Quick Review

(Refers to Dale Roberts course notes at IUPUI
&
Tutorial notes at CUHK)

The C Compilation Model



C Keywords

- They can NOT be used as identifiers.
 - ♦ Identifier: the name of variables, functions and types.

Keywords			
<code>auto</code>	<code>double</code>	<code>int</code>	<code>struct</code>
<code>break</code>	<code>else</code>	<code>long</code>	<code>switch</code>
<code>case</code>	<code>enum</code>	<code>register</code>	<code>typedef</code>
<code>char</code>	<code>extern</code>	<code>return</code>	<code>union</code>
<code>const</code>	<code>float</code>	<code>short</code>	<code>unsigned</code>
<code>continue</code>	<code>for</code>	<code>signed</code>	<code>void</code>
<code>default</code>	<code>goto</code>	<code>sizeof</code>	<code>volatile</code>
<code>do</code>	<code>if</code>	<code>static</code>	<code>while</code>

C operators

- Arithmetic: $+$, $-$, $*$, $/$, $\%$, $++$, $--$

```
int a = 10, b, c;
```

```
b = a++; /* a is now 11, b is 10 */
```

```
c = ++b; /* a, b, c are all 11 */
```

- Assignment: $=$, $+=$, $-=$, $*=$, $/=$, $\%=$

```
x += 2;
```

```
x %= 2;
```

- Relational: $>$, $<$, $>=$, $<=$, $==$, $!=$
- Logical: $\&\&$, $||$, $!$

Data Type in C

Integer	char short int int long int	unsigned signed
Floating	float double long double	
Pointer		

Functions

```
#include <stdio.h>
```

```
int sum(int x, int y){  
    return x + y;  
}
```

```
int main(int argc, char  
    **argv){  
    int a, b;  
    printf("Enter 2 numbers");  
    scanf("%d %d", &a, &b);  
    printf("%d", sum(a, b));  
    return 0;  
}
```

Notes: In standard ANSI C:

- Functions should be defined or declared *before* they are used.
- Using `int func( void );` to indicate there is no parameters for `func`.
- Local variables should be declared at the *beginning* of the function.
- Local variables will not be automatically initialized.
 - E.g, "`int a`" may contain garbage until a value is assigned.

Structure

- A collection of values (members)
 - ◆ Like a class in java or C++, but without methods.

```
struct ALL {  
    int a;  
    float b;  
};
```

- ◆ Members can be int, float,...,pointer, array, or even other structures.

Tips:

All data members should be listed in declaration, including the type and the name.

Declaration of Variables of Structure

- Declarations

method 1: declared like other variables: to declare tag (標籤字段) first, and then to declare variable.

```
struct card {  
    char *face;  
    char *suit;  
};  
struct card oneCard, deck[ 52 ],  
           *cPtr;
```

```
struct date {  
    .. .. .  
};  
struct date d1, d2, d3, d4, d5;
```

method 2: A list of variables can be declared after the right brace and use comma separated list:

```
struct card {  
    char *face;  
    char *suit;  
} oneCard, deck[ 52 ], *cPtr;
```

```
struct date {  
    .. .. .  
} d1, d2, d3;  
struct date d4, d5;
```

method 3: Declare only variables.

```
struct {  
    char *face;  
    char *suit;  
} oneCard, deck[ 52 ], *cPtr;
```

```
struct {  
    .. .. .  
} d1, d2, d3, d4, d5;
```

Accessing Members of Structures

- Accessing structure members
 - ♦ Dot (.) is a member operator used with structure variables
 - Syntax: **structure_name.member**

```
struct card myCard;  
printf( "%s", myCard.suit );
```
 - One could also declare and initialize **threeHearts** as follows:

```
struct card threeHearts;  
threeHearts.face = "Three";  
threeHearts.suit = "Hearts";
```
 - ♦ Arrow operator (->) used with pointers to structure variables

```
struct card *myCardPtr = &myCard;  
printf( "%s", myCardPtr->suit );
```

 - **myCardPtr->suit** is equivalent to **(*myCardPtr).suit**

Arrays

- Array
 - ♦ Group of consecutive memory locations
 - ♦ Same name and type, e.g.: an array of integers

- Format:

array_name [*position number*]

- ♦ First element at position 0
- ♦ **n** element array named **c**:
 - **c**[0], **c**[1] ... **c**[**n - 1**]

Example: `int my_array[12];`

`my_array[0] = -45` $\Leftarrow$ value stored

- ♦ Position number must be an integer number or an integer expression

Example: `my_array[1.5]` $\Leftarrow$ **ERROR!!**

`my_array[i+j]` $\Leftarrow$ valid if **i** and **j** are integers

`my_array[5-2] == my_array[3] == my_array[x]`

Arrays (cont'd)

- Declaring Arrays

- ♦ When declaring arrays, specify

- Name
- Type of array
- Number of elements: it must be constant or constant expression, no variables are accepted

arrayType arrayName[numberOfElements] ;

Examples:

```
/* reserve memory sufficient enough to store 100  
elements of type integer */
```

```
int c[ 100 ] ;
```

```
#define Index 6
```

```
.....
```

```
float myArray[ 3+Index ] ;
```

```
void main()  
{  
    int i=7;  
    float myArray[ i ] ;  
    .....  
}
```

X

Arrays (cont'd)

- The result?

```
#include <stdio.h>

main( void )
{
    int a[5], i;

    for( i = 0; i < 5; i++ )
    {
        printf("%8d, ", a[i]);
    }

    return 0;
}
```

```
1923535243, 14, 1003363114, 29954041, 1245120,
```

No initialization, the values of `a[i]` are all garbage. But it can pass the compiler.

Arrays (cont'd)

- Initializers

```
int n[5] = {1, 2, 3, 4, 5};
```

Example: `main()`

```
{  
    int i, a[10]={1, 2, 3, 4, 5, 6, 7, 8, 9, 10};  
    for (i=0; i<10; i++)  
        printf("Element: %d\n", a[i]);  
}
```

- ♦ If there are fewer initializations than elements in the array, then the remaining elements are automatically initialized to 0.

```
int n[5] = {0}      ✓      /* All elements 0 */  
int a[10] = {1, 2}  ✓      /* a[2] to a[9] are initialized to zeros */  
int b[5] = {1, 2, 3, 4, 5, 6} X      /* syntax error */
```

- ♦ If size omitted, initializers determine it

```
int n[ ] = { 1, 2, 3, 4, 5 }; /* 5 initializers, therefore 5 element array */
```

- ♦ **Error:** the number of elements when initialization is larger than the size of the array.

Multi-Dimensional Arrays

- 2D Arrays

- ◆ Tables with rows and columns (**m** by **n** array)
- ◆ Like matrices: specify row, then column

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Diagram illustrating the structure of a 2D array (matrix) with row and column subscripts.

Labels and arrows pointing to the corresponding parts of the array structure:

- Array name (points to 'a' in the first subscript)
- Row subscript (points to the first index, e.g., 0 in a[0][1])
- Column subscript (points to the second index, e.g., 1 in a[0][1])

Multiple-Dimensional Arrays (cont'd)

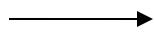
- Initialization

```
int b[2][2] = {{ 1, 2 }, { 3, 4 } };
```

```
int c[3][2] = {{ 1, 2 }, { 3, 4 }, { 5, 6 } };
```

1	2
3	4

1	2
3	4
5	6



1	2	3	4	5	6
---	---	---	---	---	---

Actual storage in the memory -
rows by rows - "row-major"

- ◆ Initializers grouped by row in braces
- ◆ If not enough, unspecified elements set to zero

```
int b[2][2] = { { 1 }, { 3, 4 } };
```

1	0
3	4

- Referencing elements

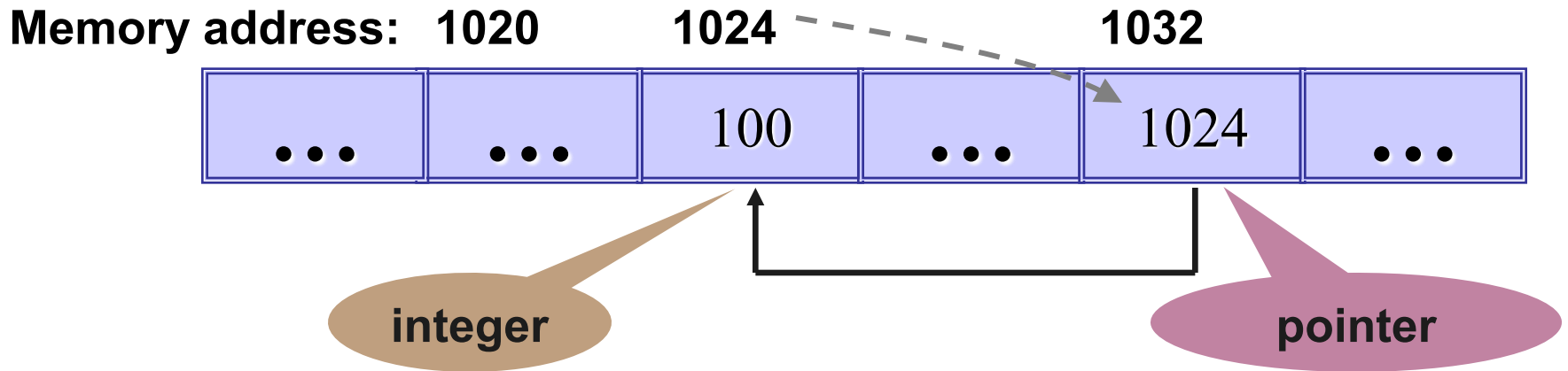
- ◆ Specify **row**, then **column**

```
printf( "%d", b[ 0 ][ 1 ] );
```


Pointers

- A pointer is a **variable** used to store the address of a memory cell/location.
- We can use the pointer to reference (引用、關聯、定位) this memory cell

Each variable is assigned a memory slot (the size depends on the data type) and the variable's data is stored there



Pointers (cont'd)

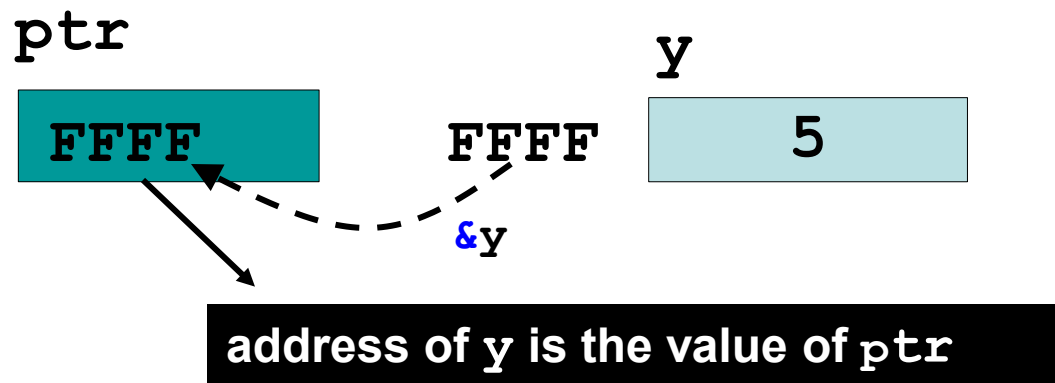
- Pointer Declaration:

```
int x, y = 5;
```

```
int *ptr; /* ptr is a POINTER to an integer variable*/
```

- Address (Reference) operator "&":

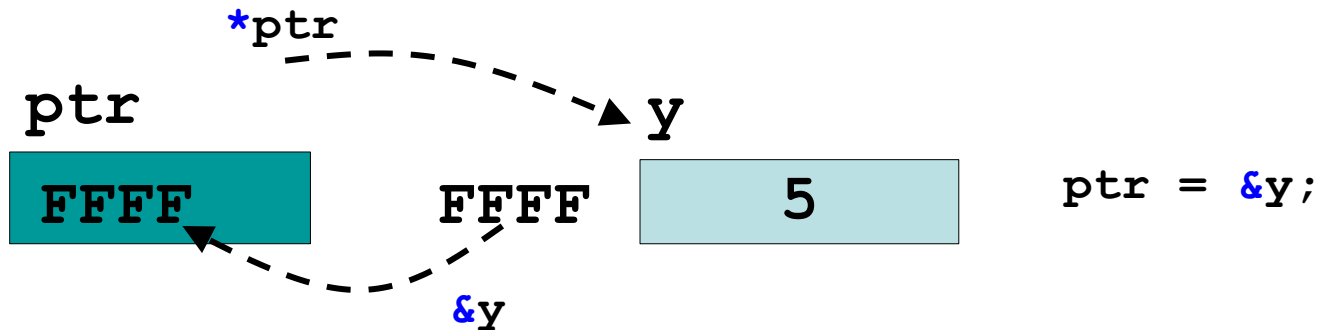
```
ptr = &y; /*assign ptr to the MEMORY ADDRESS of y.*/
```



Pointers (cont'd)

- Indirection (Dereference) operator “*****”:

`x = *ptr; /* assign x to the value what ptr is pointed to */`



Returns a synonym/alias of what its operand points to
***ptr** returns **y** (because **ptr** points to **y**)

`x = *ptr; //right-value`
`*ptr = 7; //left-value`

x

5

y

7

Pointer Variable Initialization

- ◆ Initialize pointers to 0, **NULL**, or an address
 - 0 or **NULL** – points to nothing (NULL preferred)
 - Example:

```
int *ptr, x;
```



```
x = 10;
```



```
ptr = &x;
```



```
*ptr = ?;
```

Question:

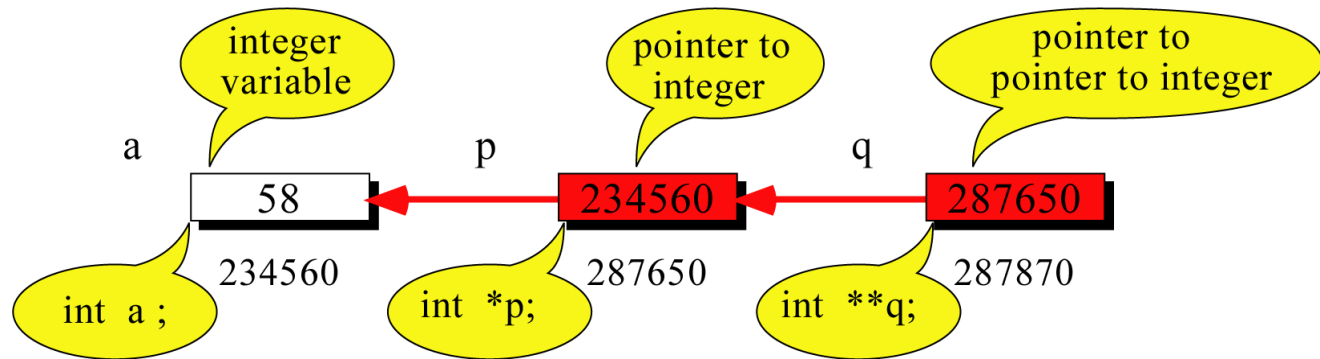
```
int y;  
y = 12 - *ptr;
```

Pointer to Pointer



```
// Local Declarations
```

```
int    a ;  
int    *p ;  
int    **q ;
```



What is the output?

a =
*p =
**q =

?

```
// Statements
```

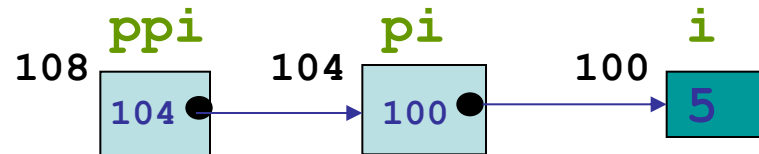
```
a = 58 ;  
p = &a ;  
q = &p ;
```

Pointer to Pointer

Example:

```
int i,*pi,**ppi;  
i = 5;  
pi = &i;  
ppi = &pi;
```

Variable	Address	Value at address
i	100	5
pi	104	100
ppi	108	104



Usage	Meaning	Value
pi	address of <code>int</code>	100
*pi	<code>int</code> value	5
&pi	address of pointer	104
ppi	address of pointer	104
*ppi	address of <code>int</code>	100
**ppi	<code>int</code> value	5

sizeof Operator

- **sizeof**

- ◆ Returns size of operand in bytes
- ◆ For arrays: size of (one element * number of elements)
- ◆ if **sizeof(int)** equals 4 bytes, then

```
int myArray[ 10 ];
```

```
printf( "%d", sizeof( myArray ) );
```

will print 40

- **sizeof** can be used with

- ◆ Variable names
- ◆ Type name
- ◆ Constant values

sizeof Operator (cont'd)

- **sizeof** is a compile-time operator, so it does not incur any execution-time overhead.
- The number of bytes used to store a particular data type may vary between systems. You can use **sizeof** to check the bytes used to store a kind of data in your system.

Storage Management

- C supports 4 functions, **malloc()** , **calloc()** , **free()** , and **cfree()** for storage management
- **malloc(n)** :
 - allocate a node while its content is still 'garbage'
 - **n** is an integer, indicating the size of memory in byte which you would like to allocate
 - **malloc()** return a pointer to that memory: [void *]
 - So, you have to use cast operator (**type**) , to change the type of the pointer.

Example:

```
int *ip;  
ip = (int*) malloc(sizeof(int));  
struct treeNode *tp;  
tp = (struct treeNode *) malloc(sizeof(struct  
treeNode));
```

Storage Management (cont'd)

- **free(p) :**
 - ♦ **free()** will release the memory allocated by **malloc()**.
 - ♦ **p** is the pointer containing the address returning from **malloc()**.
(It can be **NULL**.)

Example:

```
int *ip;  
ip = (int*) malloc(sizeof(int));  
... ..  
free(ip);
```

Example:

```
struct treeNode *tp;  
tp=(struct treeNode *)malloc(sizeof(struct treeNode ));  
... ..  
free(tp);
```

Storage Management (cont'd)

- ◆ When there is no further memory, **malloc()** will return **NULL** pointer. It is a good idea to check the returning value of **malloc()** .

```
if ((ip=(int *)malloc(sizeof(int))) == NULL) {  
    printf("\nMemory is FULL\n");  
    exit(1);  
}
```

- ◆ When you free the memory, you must be sure that you pass the original address returning from **malloc()** to function **free()** . Otherwise, system exception may be happened

```
int *ip;  
ip = (int*) malloc(sizeof(int));  
... ..  
free(ip); /* If you free(ip) after ip++, then  
           it's wrong! */
```

Storage Management (cont'd)

- **calloc(n, size) :**

- ♦ **calloc()** allow you to allocate an **n** elements array of same data type. Because **n** can be an integer variable, you can use **calloc()** to allocate a dynamic size array.
- ♦ **n** is the element number of array that you want to allocate.
- ♦ **size** is the number of byte of each element.
- ♦ Unlike **malloc()**, **calloc()** guarantees that memory contents are all zero

Example: allocate an array of 10 elements

```
int *ip;
```

```
ip = (int*) calloc(10, sizeof(int));
```

***(ip+1)** refer to the 2nd element, the same as **ip[1]**

***(ip+i)** refer to the i+1th element, the same as **ip[i]**

Storage Management (cont'd)

- **calloc(n, size) :**
 - ♦ Like **malloc()**, **calloc()** will return **NULL**, if no further memory is available.
- **cfree(p) :**
 - ♦ **cfree()** releases the memory allocated by **calloc()**.

Example:

```
cfree(ip) ;
```

C Programming Style

- Make your program elegant and **readable**
 - ◆ Include a brief header block at the top of each function

```
/******
```

```
* The function name
```

```
* Description of the function purpose
```

```
* variables' name and brief explanation
```

```
***** /
```

- ◆ Use inline comments sparingly but whenever necessary
- ◆ Follow standard formatting conventions

Indentation

braces

```
.....
int withdraw (int requestedAmount) {
if (balance < requestedAmount) {
    return 0;
}                                /* end if */
else {
    balance = balance - requested amount;
    printf("Withdrawal of $" %4.2f
           successful, leaving %4.2f.",
           requestedAmount, balance);

    return 1;
}                                /* end else */
}                                /* end function */
```

C Programming Style

- Naming conventions
 - ◆ Constants: the entire word is capitalized
for example: `const PI = 3.14159;`
 - ◆ Variables
 - Hungarian notation
for example: `int iSize; float * pElement;`

Others... You can find a lot of materials

Whice ONE is the best? **No answer**

Clear, Readable, Consistent

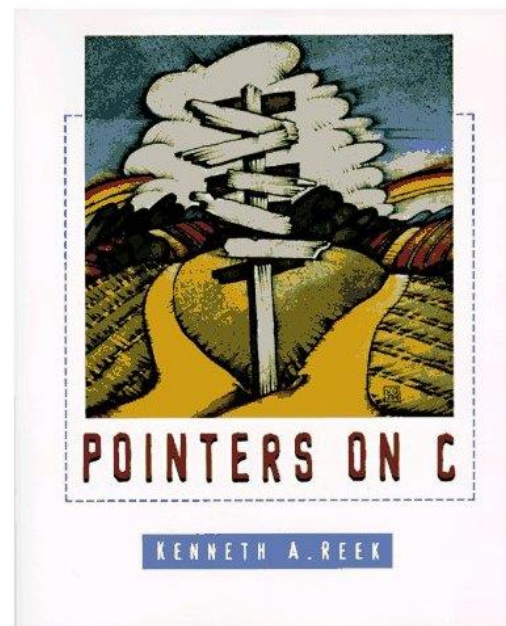
Reference

Pointers on C

Kenneth A. Reek

Addison Wesley, 1997

ISBN: 0-673-99986-6



Chinese Version: C和指針

徐波 譯

人民郵電出版社, 2008